700105 Simulation & Concurrency

UNIVERSITY of HULL

What to put in the lab books

- Join GitHub Classroom
  - https://classroom.github.com/a/IeDBPXHF
  - Ideally your work should be presented in markdown – just to make you aware of this for things like setting up a portfolio in GitHub
- Relevant code
  - You can also use this repository as your actual code repository
- Results of tests where appropriate
- Demonstrate that you understand what the code does
  - Description of the techniques used in the code
  - Include diagrams where appropriate

Lab book 1 – Thursday 12th Feb

- **Q1 Sphere Sphere Intersection Test (Formative – not marked)**
- **Q2 Closest Distance from a Point to a Line**
- **Q3 Sphere Line Intersection Test**
- **Q4 Closest Distance from a Point to a Plane**
- **Q5 Sphere to Plane Collision (Formative – not marked)**

Lab book 2 – Thursday 19th Feb

- **Q1 Be able to load and unload difference scenarios easily**
- **Q2 Be able to render primitive shapes**
- **Q3 Be ready to simulation to the sandbox**

Story So Far

Intersection

- We started to look at how we can tell if two shapes have collided
- Last time we looked at integrating velocities and positions for objects that are accelerating due to a force

Sphere Sphere Intersection

Sphere Sphere Intersection

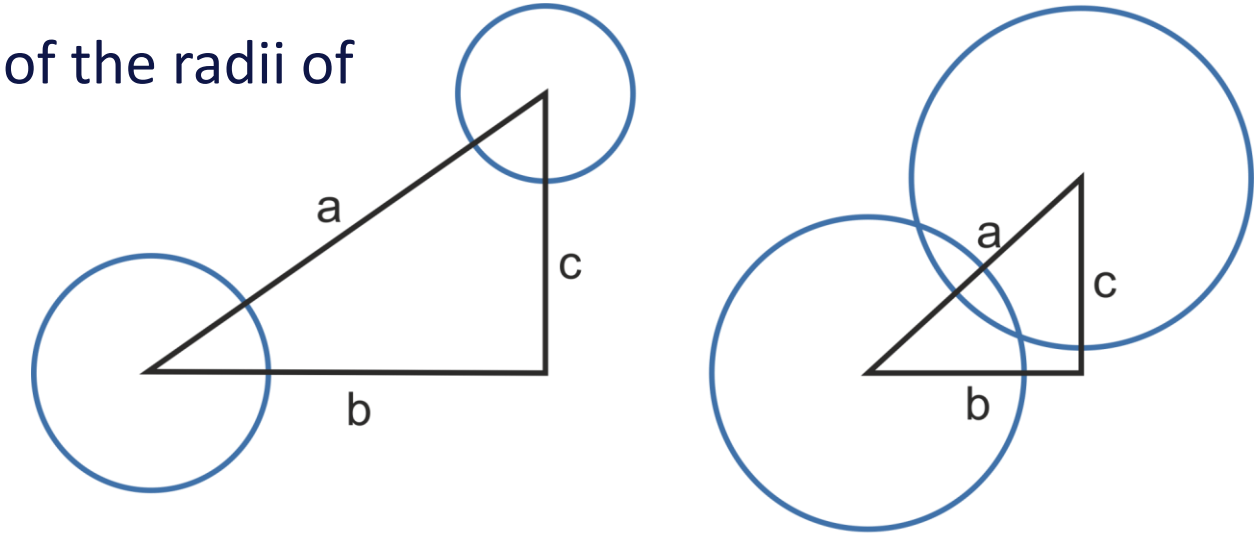
Get the distance between two sphere points

If that distance is less than the sum of the radii of the spheres they are intersecting

Square root operation is slow

Get the ***squared*** distance between two sphere points

If that distance is less that the ***squared*** sum of the radii of the spheres they are intersecting



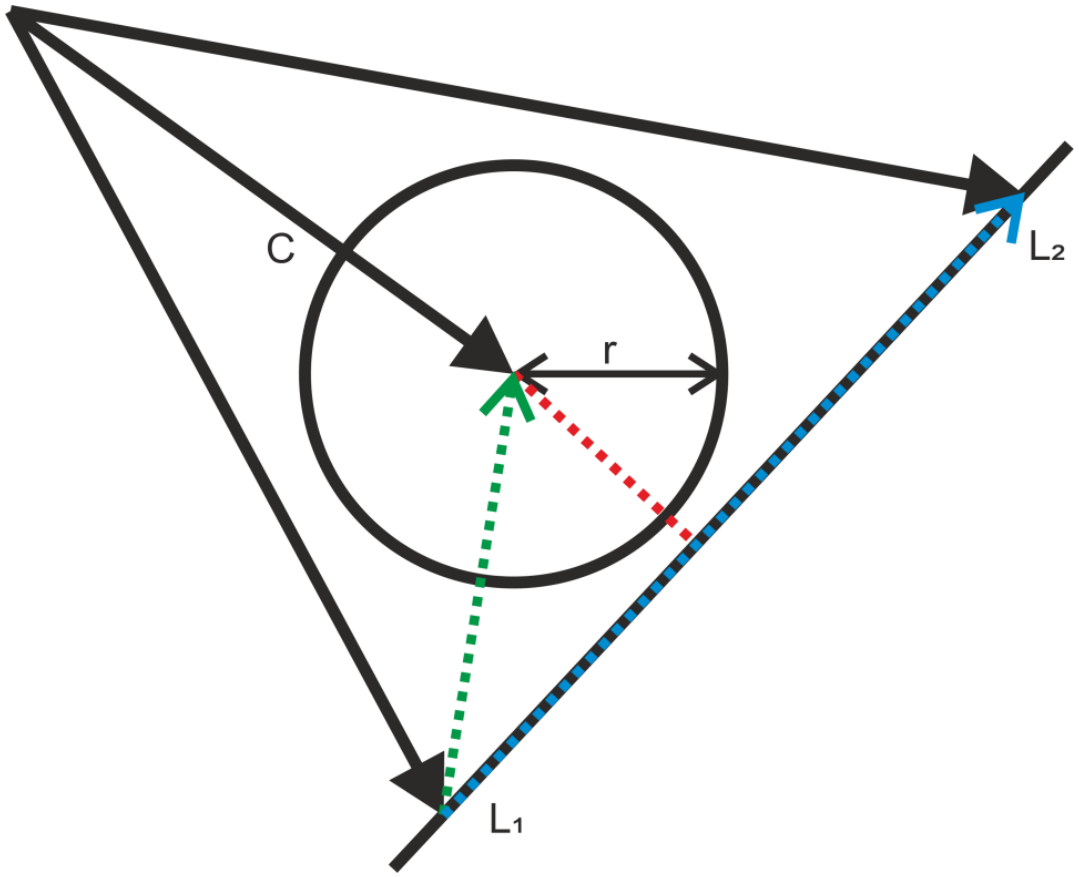
Pythagoras's Theorem

$a^2 = b^2 + c^2$

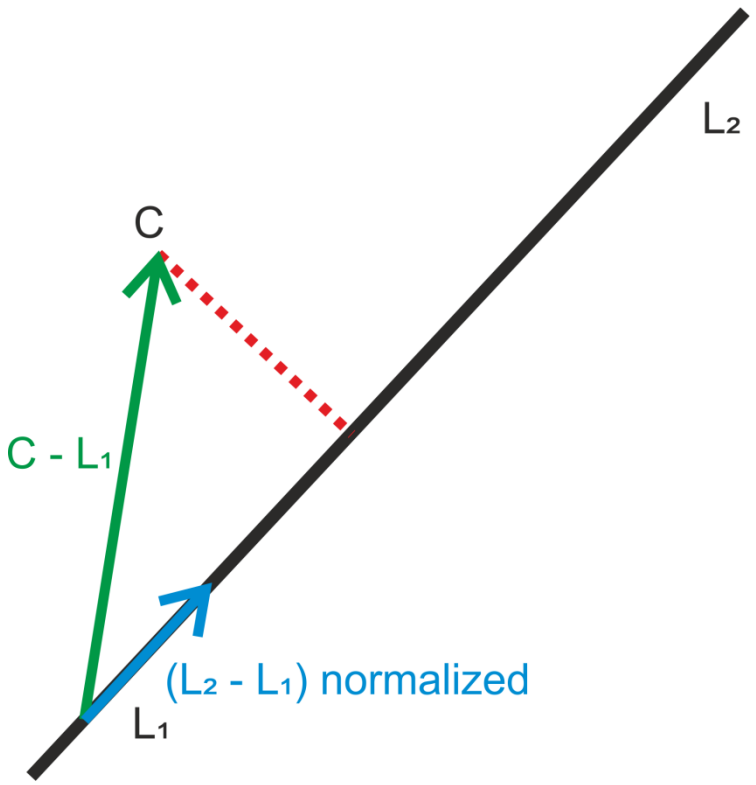
Sphere Cylinder (and Line, and Line Segment) Intersection

Closest Point to a Line – Geometric Method

- C - L1
- L2 – L1

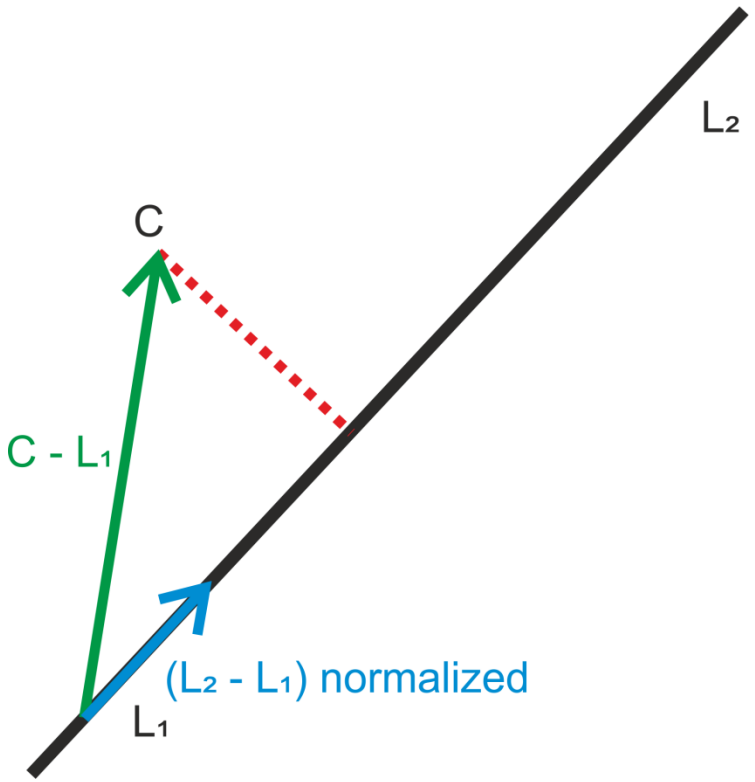


Closest Point to a Line – Geometric Method



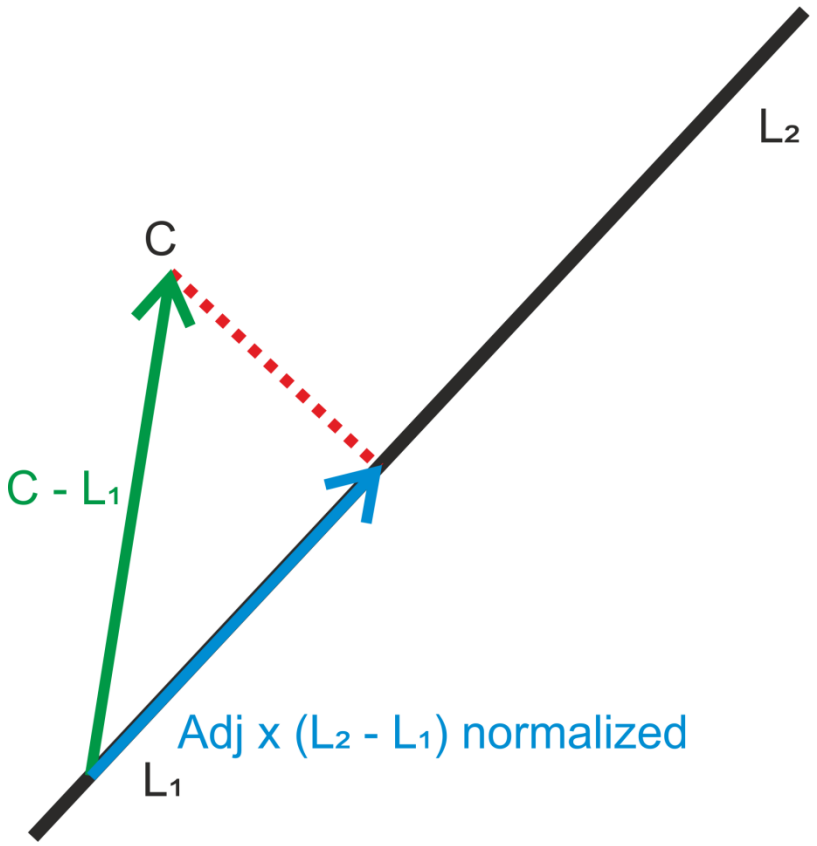
- Normalise L2 - L1 and dot with C - L1
- $\boldsymbol{a}.\boldsymbol{b} = |\boldsymbol{a}||\boldsymbol{b}| \cos\theta$
- $|\boldsymbol{a}|$=1 so $\boldsymbol{a}.\boldsymbol{b} = |\boldsymbol{b}| \cos\theta$
- $Hyp \times \cos\theta = Adj$

Closest Point to a Line – Geometric Method



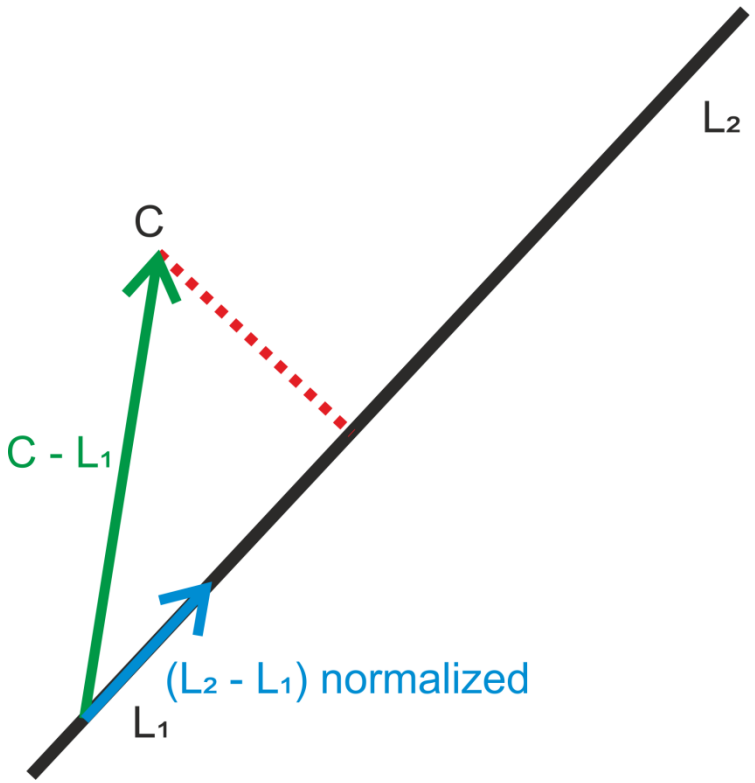
- Note from here you ***could*** calculate the opposite red length using trigonometry
- $\text{Hyp} \times \sin\theta$

Closest Point to a Line – Geometric Method



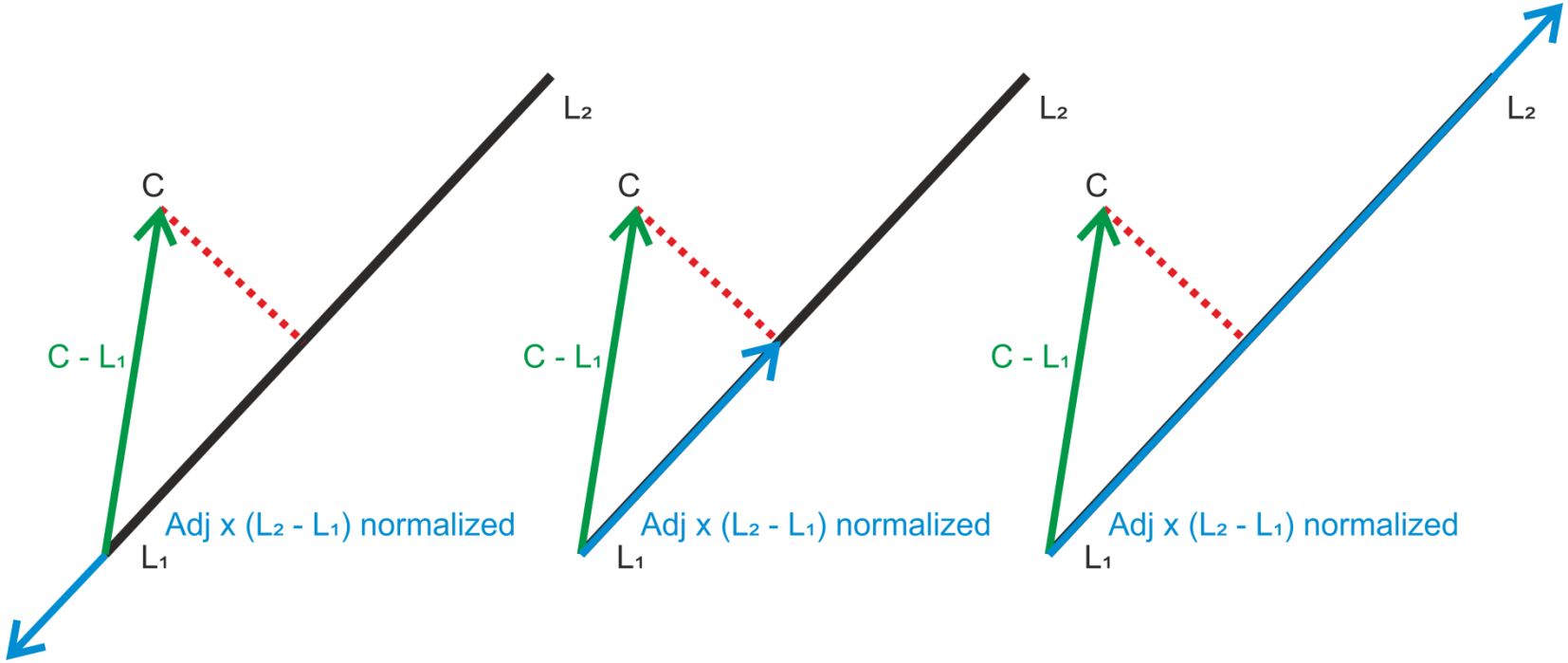
- $Hyp \times \cos\theta = Adj$
- $C - L1 - \ Adj \times (L_2 - L_1)\ normalised$
- Know the length of the adjacent side is useful in the next section

Closest Point to a Line – Geometric Method



- By normalising the direction of the line we ensure that our calculations are in units of length (e.g. metres)
  - We calculate the number of the vectors that "fit" in the space
- If our direction has a length of "distance travelled in one second" (i.e velocity) then the result can be thought of as time (until intersection – for example)

Closest Point to a Line***Segment*** – Geometric Method



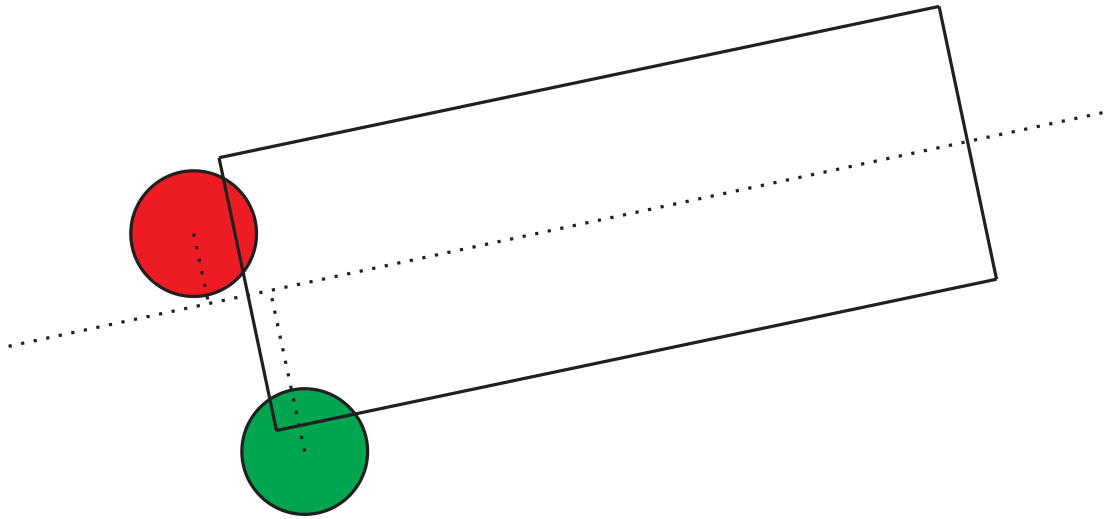
- If Adj is negative – no collision
- If Adj is greater than length of (L2 – L1) – no collision

Sphere ~~Cylinder~~ Cylindrical Tube

- To test a Sphere against a ~~Cylinder~~ cylindrical tube simple compare the shortest distance to the line with the sum of the two radii instead (the radius of the sphere and the radius of the cylinder)

Sphere Cylinder Ends

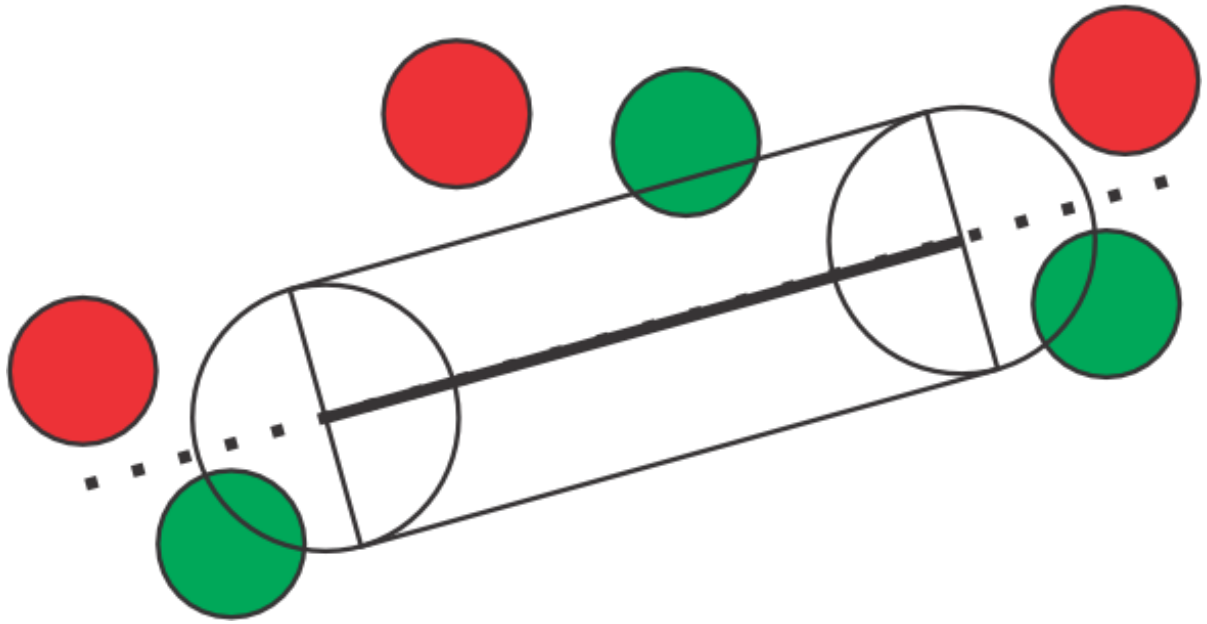
- To capture this case we can extend the cylinder end points, along the line, by the radius of the sphere
- Note that the normal vector of the collision will be different for this surface
  - Often if find a collision we return the collision normal to save having to figure out how the collision happened



Sphere Capsule Intersection

Sphere Capsule

- Conceptually capsule can be modelled as a cylindrical tube with a sphere at each end
- Programmatically you just do the end cap tests in a different way



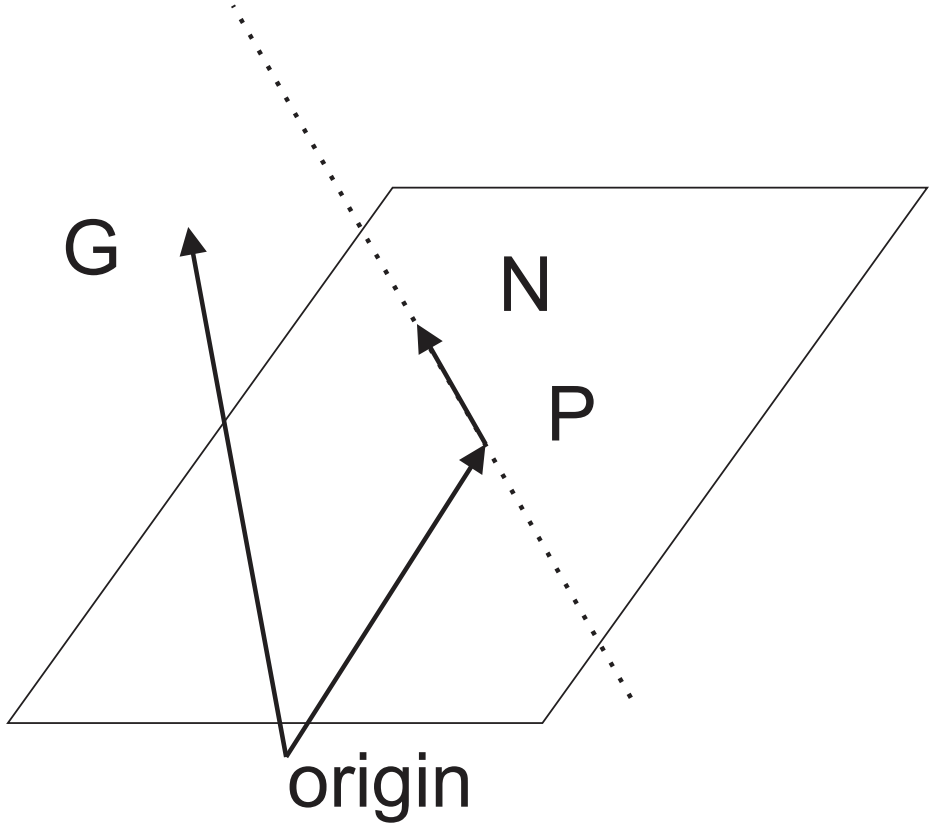
Sphere Plane Intersection

Defining the plane

- Plane can be defined
  - A point and a normal vector
  - Three points in the plane
  - You probably want to store all 4 vectors
  - You should ensure that your plane is valid, and that your normal vector is unit

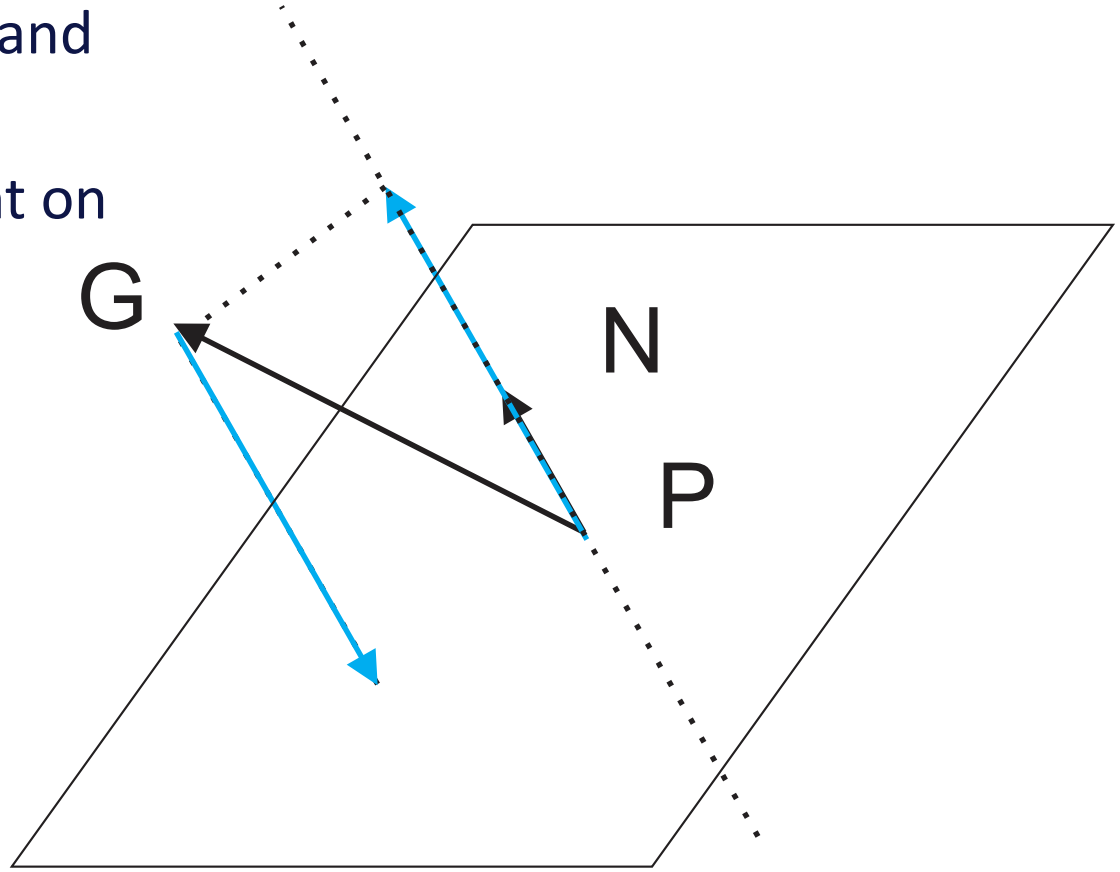
Closest Point to a Plane – Geometric Method

- Plane represented by point P and normal vector N
- We want to closest (i.e. perpendicular) distance from general point G to the plane



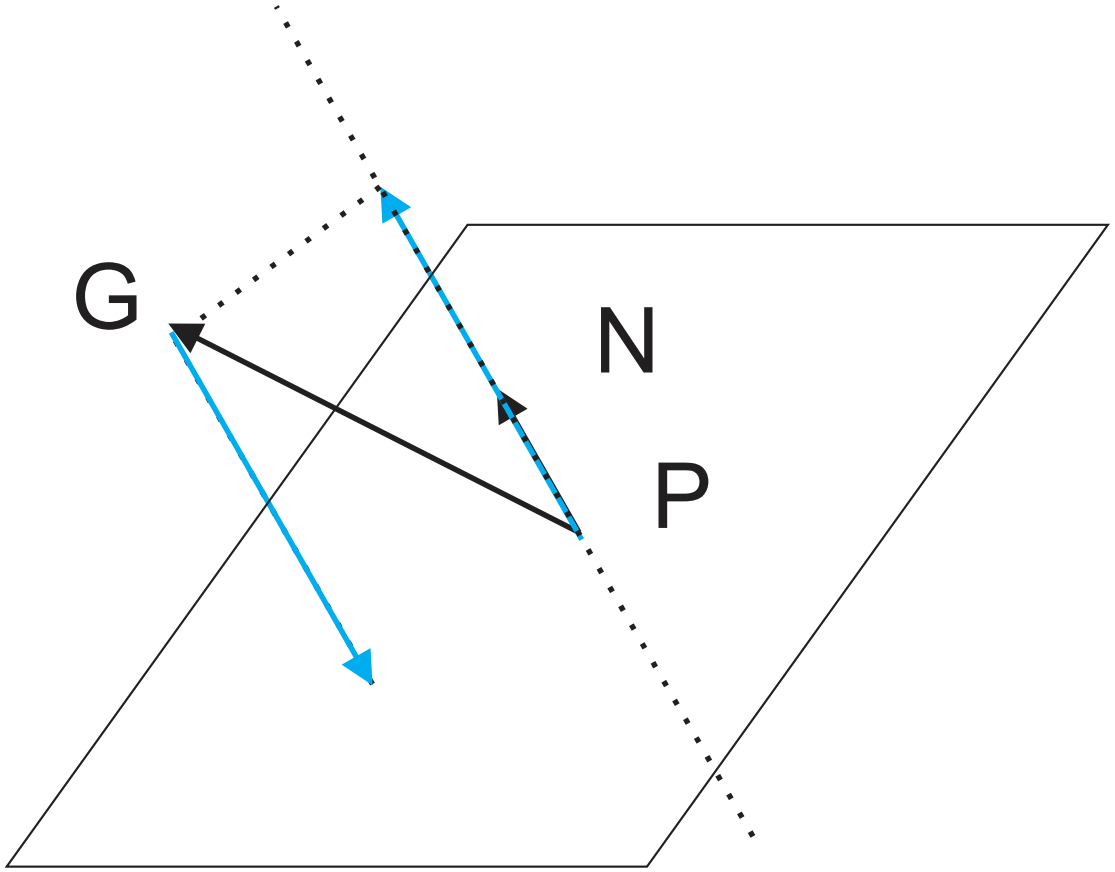
Closest Point to a Plane – Geometric Method

- First get closest point on the line formed by P and N to the general point G
- Calculate the vector from P to the closest point on the line to G
- Subtract that vector from G



Sphere Plane Intersection

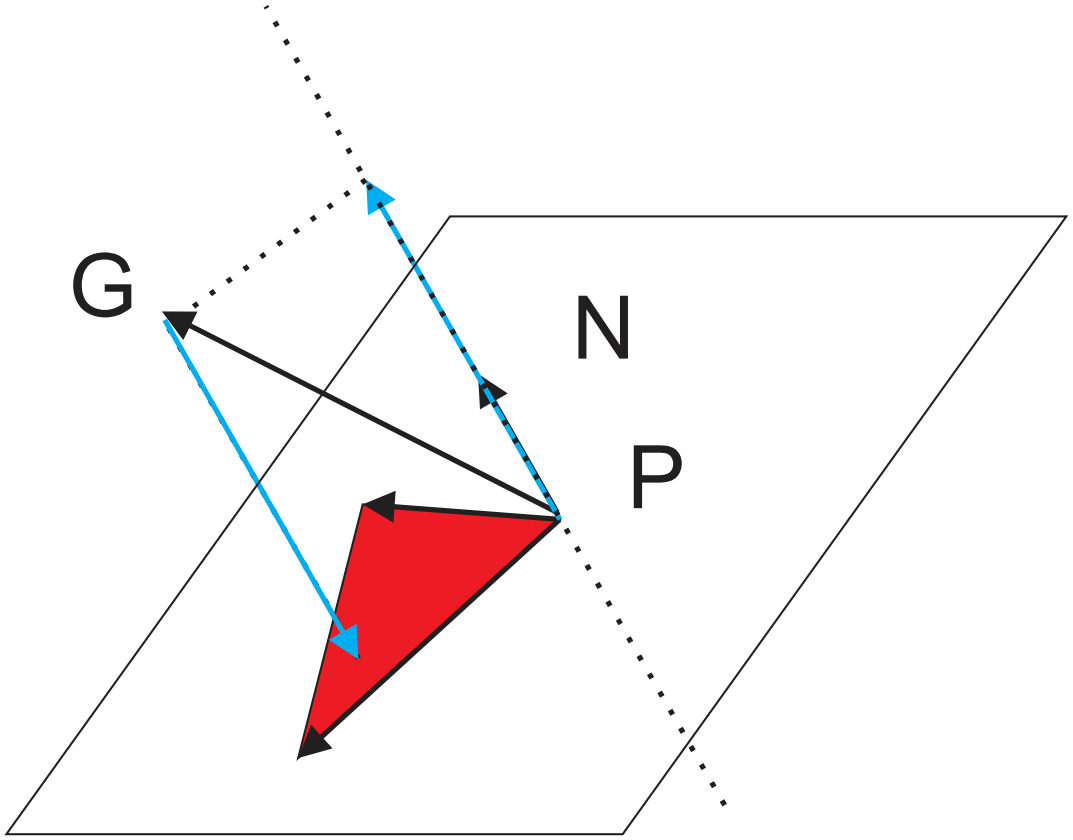
- G is the point at the centre of the sphere
- If the closest point on the plane is less that the radius of the sphere there is an intersection



Sphere Triangle Intersection

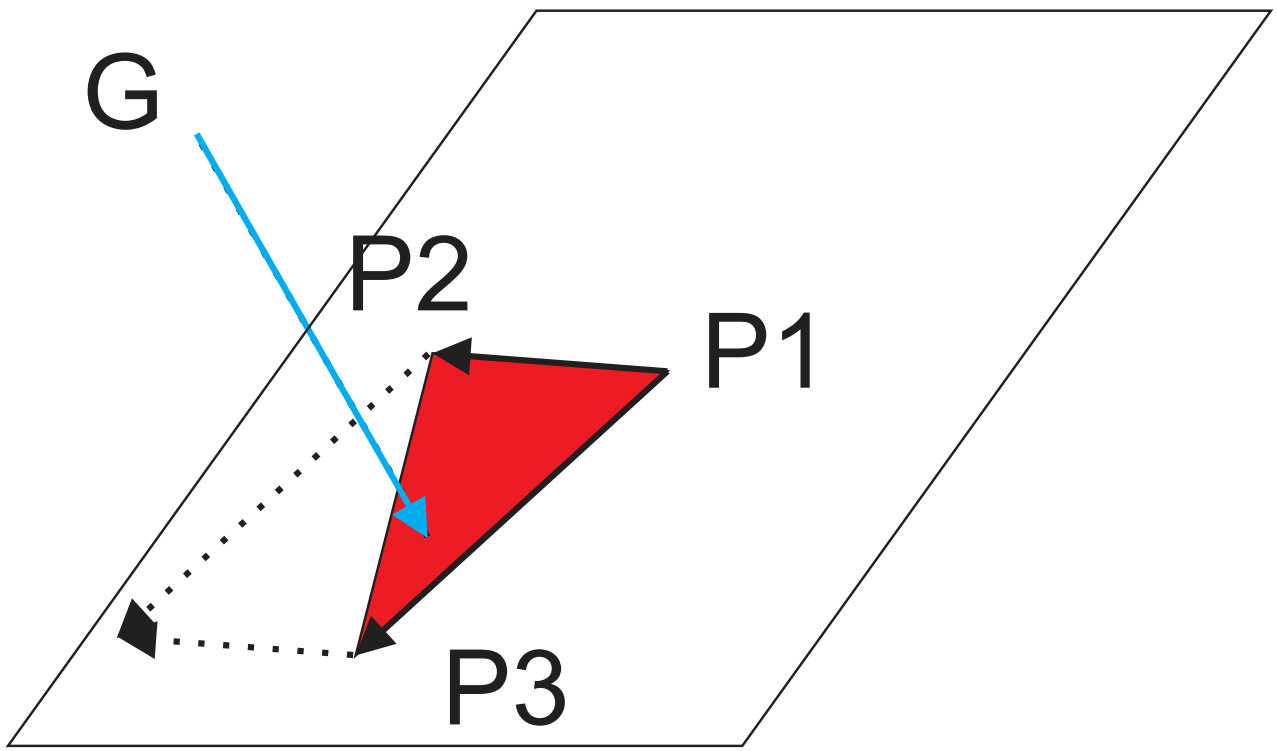
Same technique again

- First find closest point on the plane (that the triangle lies in) to the point
- Can calculate the normal using the cross product
- Then we have a 2D problem



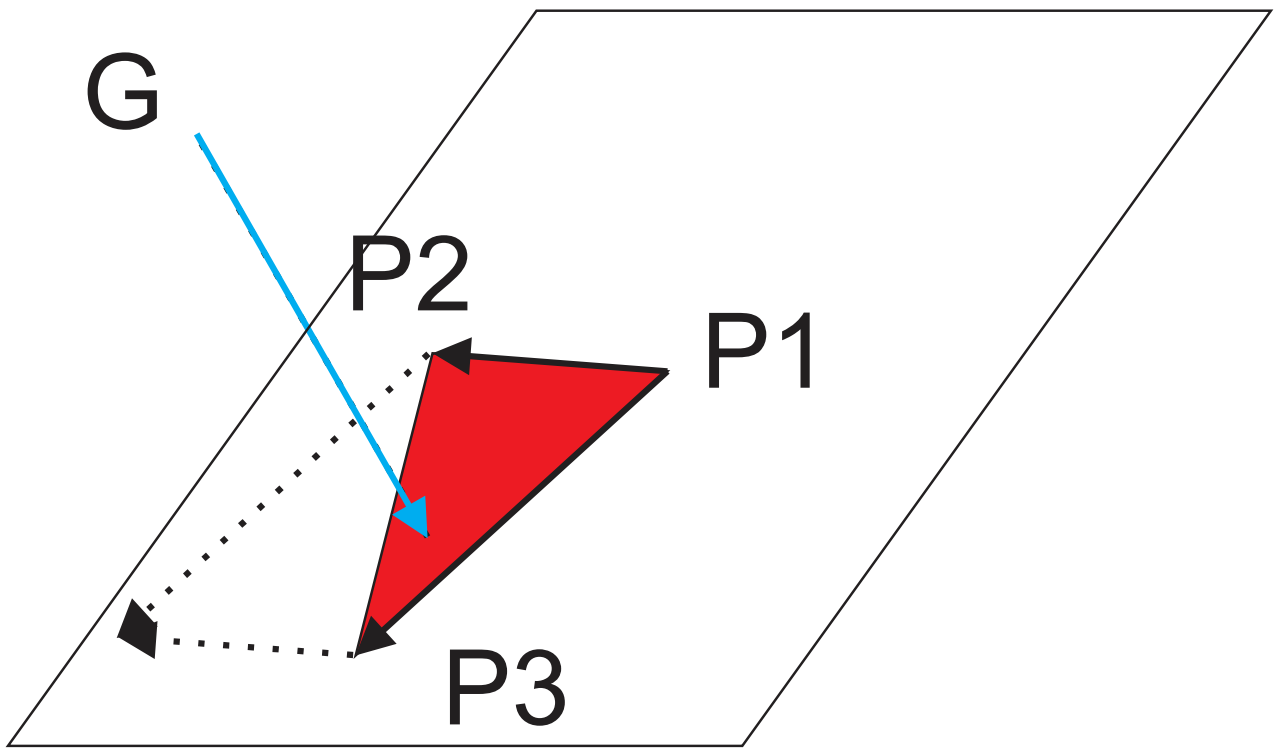
Same technique again

- Find the distance from the closest point to G on the plane and the line segment created by P1->P2
- Do the same for the line segment P1->P3
- It is useful to get to point in terms normalised values with respect to the length of each sides.
- If the sum of those values is more than 0 and less that 1 the point is in the quad
- If the sum is more than 0 and less that 0.5 the point is in the triangle



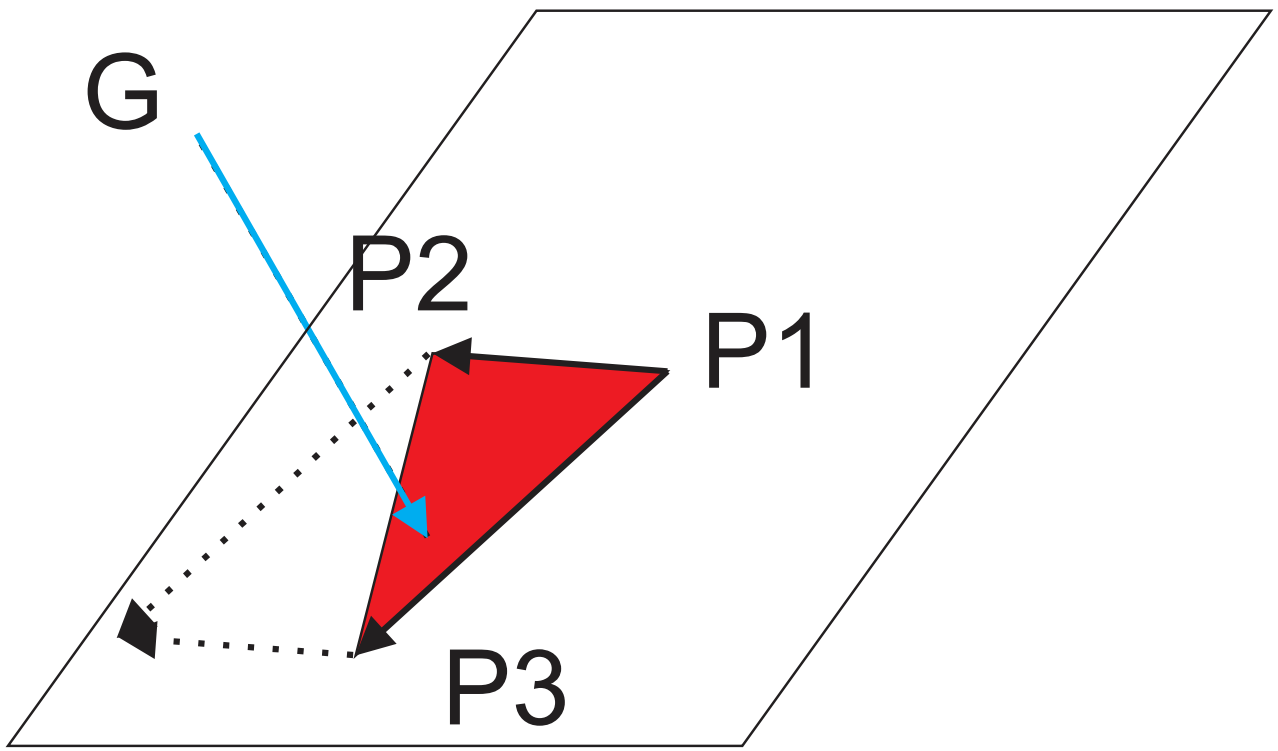
Same technique again

- Find the distance from the closest point to G on the plane and the line segment created by P1->P2
- Do the same for the line segment P1->P3
- It is useful to get to point in terms normalised values with respect to the length of each sides (Barycentric Coordinates)
- If the sum of those values is more than 0 and less that 1 the point is in the quad
- If the sum is more than 0 and less that 0.5 the point is in the triangle



Sphere Triangle Intersection

- If the projected point is in the triangle check that distance against the radius of the sphere
- Otherwise check the line segment on each edge
- And the point at each corner
- Intersection tests with triangle meshes are rare – and often we'll instead use some sort of bounding volume



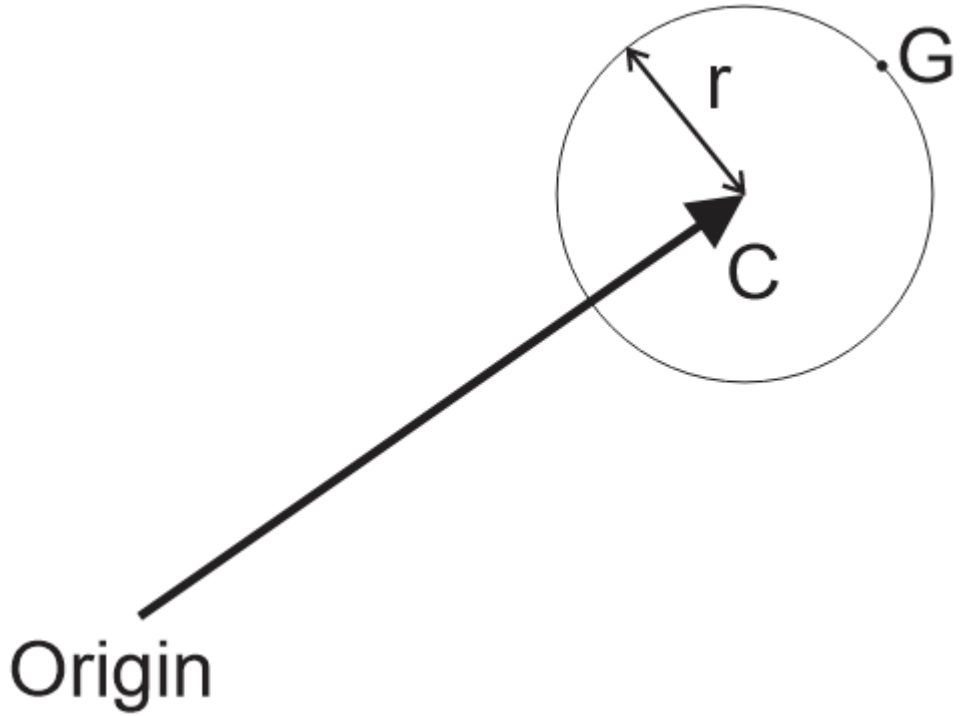
Extra Maths – parametric / quadratic methods

Representing Shapes

- The Implicit for is a Boolean function that represents all the points on a surface
  - $(x - x_p)^2 + (y - y_p)^2 + (z - z_p)^2 = R^2$
  - $(x - x_p)^2 + (y - y_p)^2 + (z - z_p)^2 - R^2 = 0$ represents the same sphere
  - This sort of representation is particularly useful for things like inside outside tests
- We can also represent shapes in a parametric form
  - $x = x_p + u$
  - $y = y_p + v$
  - $z = z_p \pm \sqrt{R^2 - u^2 - v^2}$
  - $u^2 + v^2 \leq R^2$

Sphere Equation

- A sphere with centre **C = (C_x, C_y, C_z)** and radius **r**:
- A general point G_S is on the surface of the sphere if $|G_S - C|^2 = r^2$
- Expanded:
- $(G_{Sx} - C_x)^2 + (G_{Sy} - C_y)^2 + (G_{Sz} - C_z)^2 = r^2$

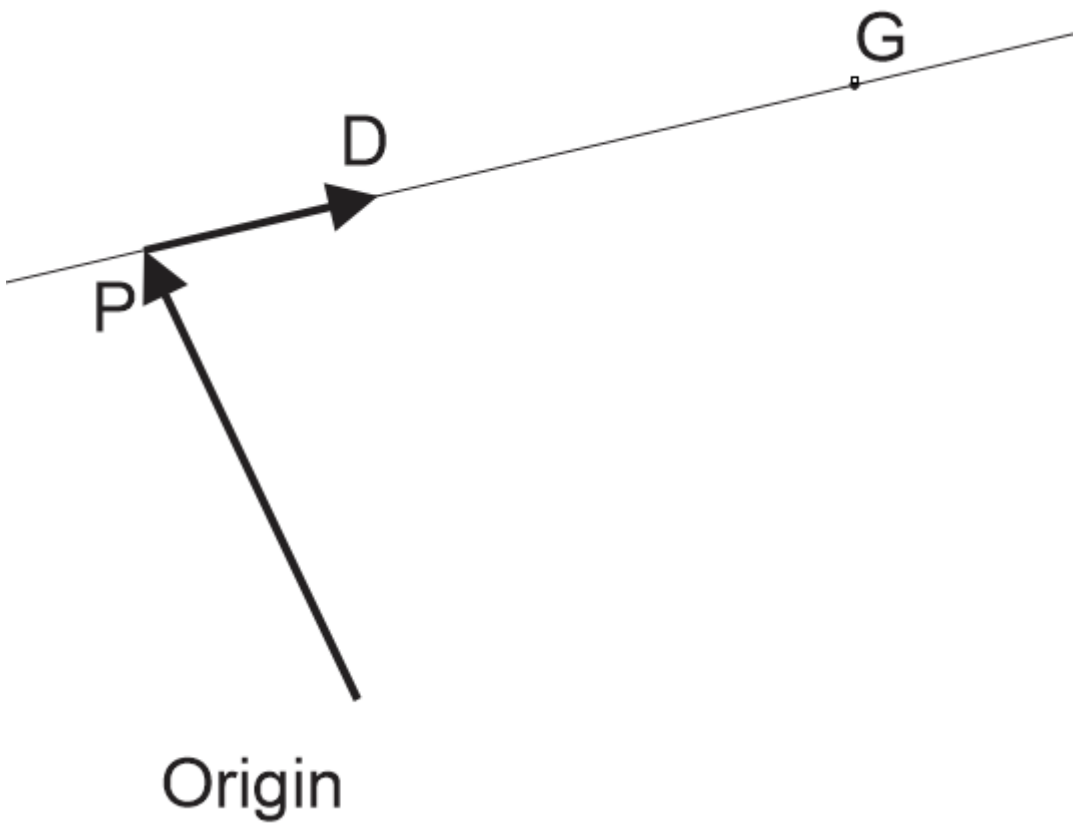


Line Equation

- A line starting at point $\mathbf{P_L}$ with direction **D**:
- $G_L = \mathbf{P_L} + t\mathbf{D}$

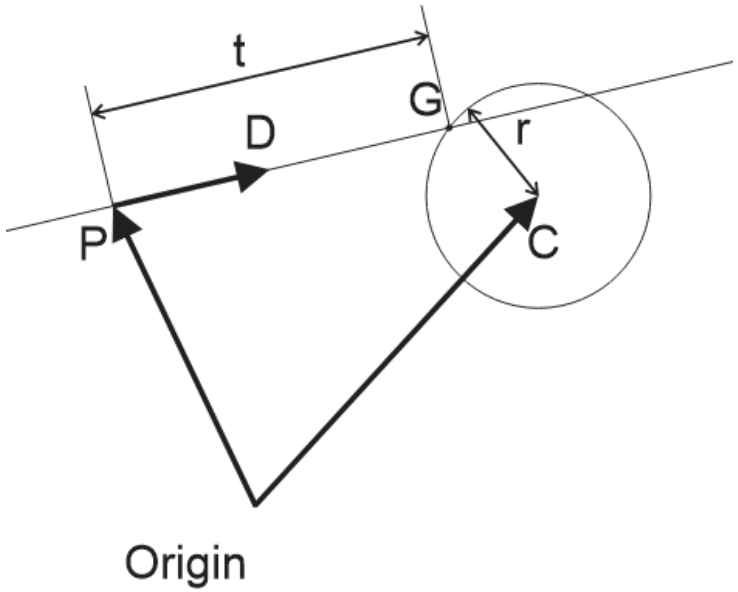
Where:

- $\mathbf{P_L}$ = a point on the line ($\mathbf{P_{L\,x}}$, $\mathbf{P_{L\,y}}$, $\mathbf{P_{L\,z}}$)
- D = direction vector (D_x, D_y, D_z)
- t = distance along the line



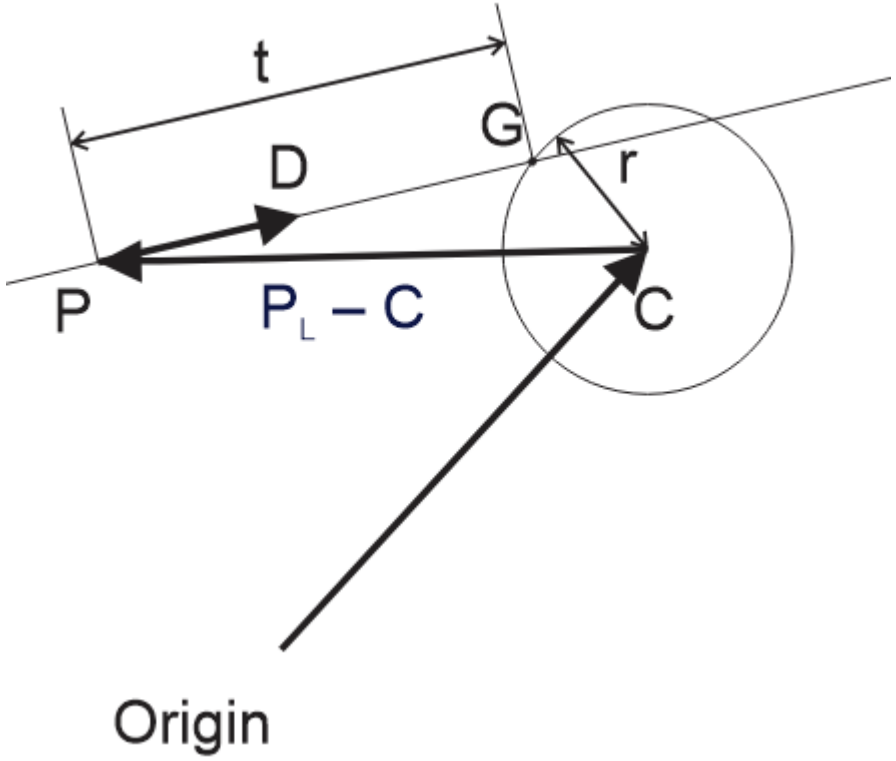
Key Concept

- If the line hits the sphere, then **some value of t** produces a point that satisfies the sphere equation.
  - $G_L = G_S$
- So we substitute the line equation into the sphere equation.
- Substitute G_L into $|G_S - C|^2 = r^2$:
- $|P_L + tD_L - C|^2 = r^2$
- This is now an equation with only **t** unknown.



Simplify the Expression

- $|P_L + tD_L - C|^2 = r^2$
- Let $L = P_L - C$
- $|L + tD_L|^2 = r^2$
- I like to think of this step as translating the line into "Sphere Space"



Expand and Rearrange

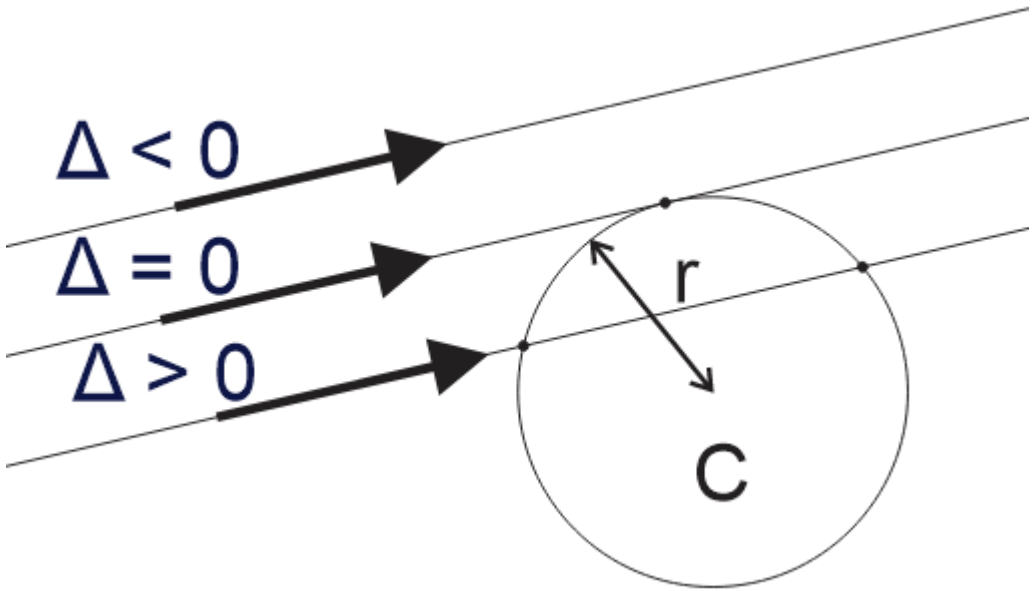
- $|L + tD_L|^2 = r^2$
- $(L + tD) \cdot (L + tD) = r^2$
- $L \cdot L + 2t(D \cdot L) + t^2(D \cdot D) = r^2$
- $(D \cdot D)t^2 + 2(D \cdot L)t + (L \cdot L - r^2) = 0$
- Now this is in the quadratic form $at^2 + bt + c = 0$

Solving the quadratic

- $a = D \cdot D$
- $b = 2(D \cdot L)$
- $c = L \cdot L - r^2$
- $t = (-b \pm \sqrt{(b^2 - 4ac)}) / (2a)$ <- formula to find roots of quadratic

The Discriminant Δ

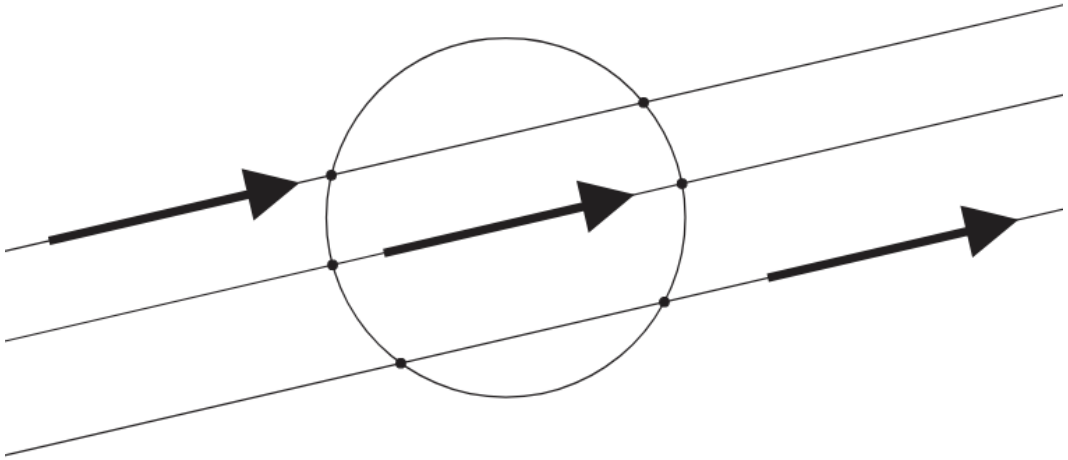
- Let $\Delta = b^2 - 4ac$
- $\Delta < 0$
  - no intersection
- $\Delta = 0$
  - Only one root (or both roots are the same?)
  - Line is tangential to sphere (touches once)
- $\Delta > 0$
  - two intersections (enters and exits)



The Discriminant Δ continued

- $\Delta > 0$
  - two intersections (enters and exits)
- Roots represent the number of D_L's down the line that the intersection occurs
  - This has implications based on what the units of D_L are

- If both roots are positive -> both intersections occur on the line in front of G_L
  - Usually keep the smallest one
- If one is negative and one is positive -> G_L is inside sphere
- If both roots are negative -> both intersections occur on the line behind G_L
  - Usually ignore both



Summary

Determinant Δ	Roots	Interpretation
negative	No roots	The line misses the object
zero	There is one root	The line touches the object
positive	There are two roots	The line enters and exits the object
If there are any roots then		
	If root is positive	Root occurs ahead of G_L
	If root is negative	Root occurs behind of G_L

Applying to other shapes - plane

- Plane defined by a Point on plane P_o and a Normal vector: N
- Plane equation $N \cdot (P - P_o) = 0$
- Line equation $G_L = \mathbf{P_L} + t\mathbf{D}$
- Substitute P with G_L
- $N \cdot ((\mathbf{P_L} + tD) - P_o) = 0$
- $N \cdot (\mathbf{P_L} - P_o) + t\,(N \cdot D) = 0$
- Rearrange and solve for t
  - Only ever one intersection – so no quadratic

Time to “Code”

Canvas Workshop 3.1 – creating some test scenarios

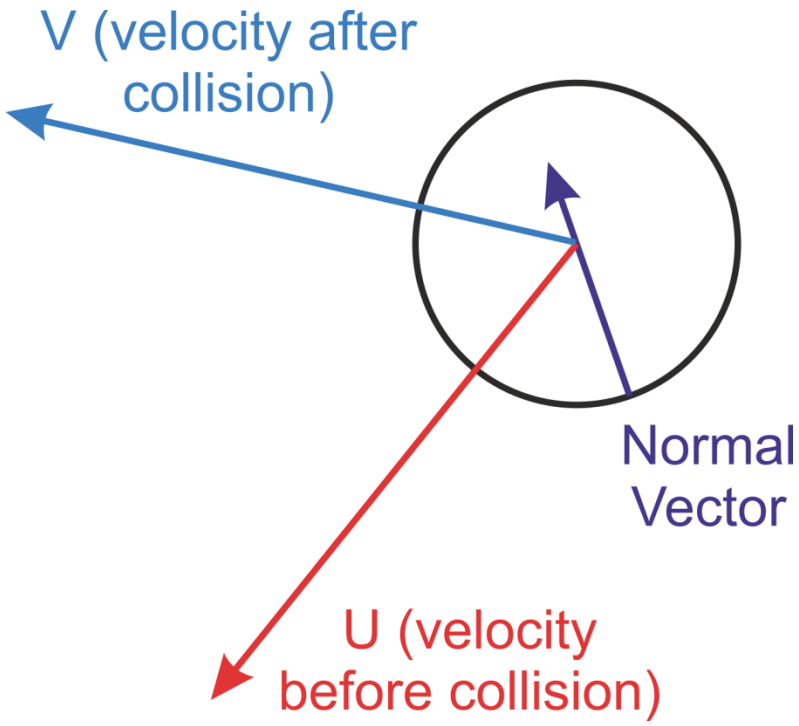
Collision Response

Newton's Laws

- 1st law - An object at rest stays at rest and an object in motion stays in motion with the same speed and in the same direction unless acted upon by an unbalanced force
- 2nd law- Force = mass x velocity / time
  - Force = mass x acceleration
- 3rd law - Every action has an equal and opposite reaction
- $F_g = G\ m_1\ m_2 / r^2$
  - Remember though we like to cut corners
  - Force = mass x acceleration where acceleration is 9.81 ms^{-2}

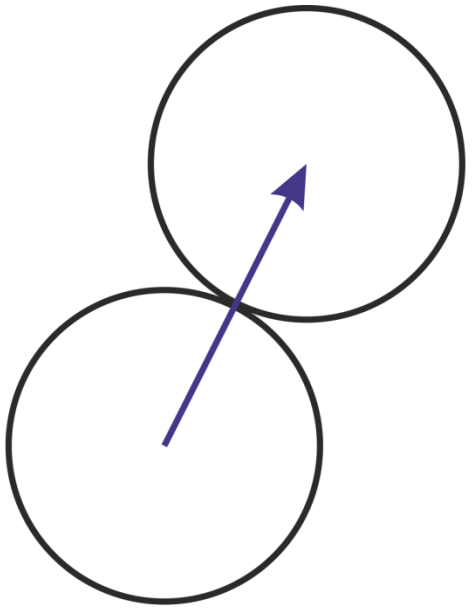
Colliding with “fixed” objects

- When colliding with fixed objects (assuming elastic collisions and no friction) we just need to “reflect” the velocity of the moving object in the direction of the normal vector at the collision point



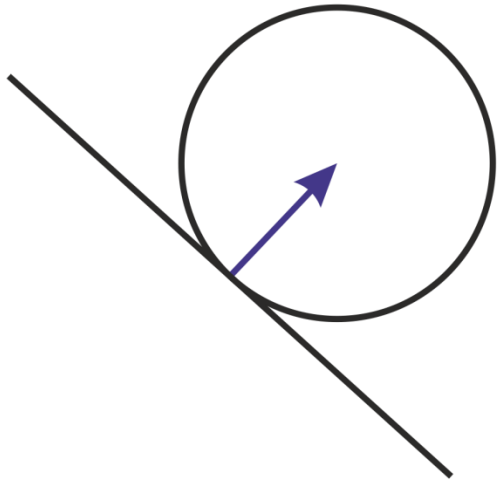
Getting the collision normal

- The best way to get the Normal Vector depends on the collision



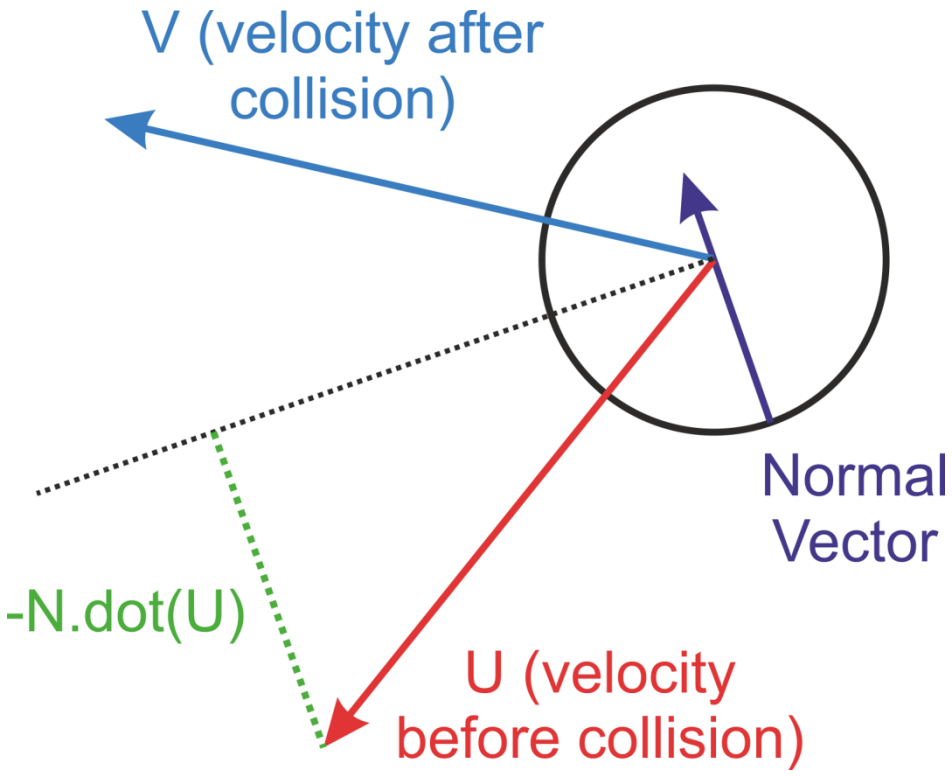
N = (P1 - P2).Normalized()

N = (P1 - CollisionPoint).Normalized()



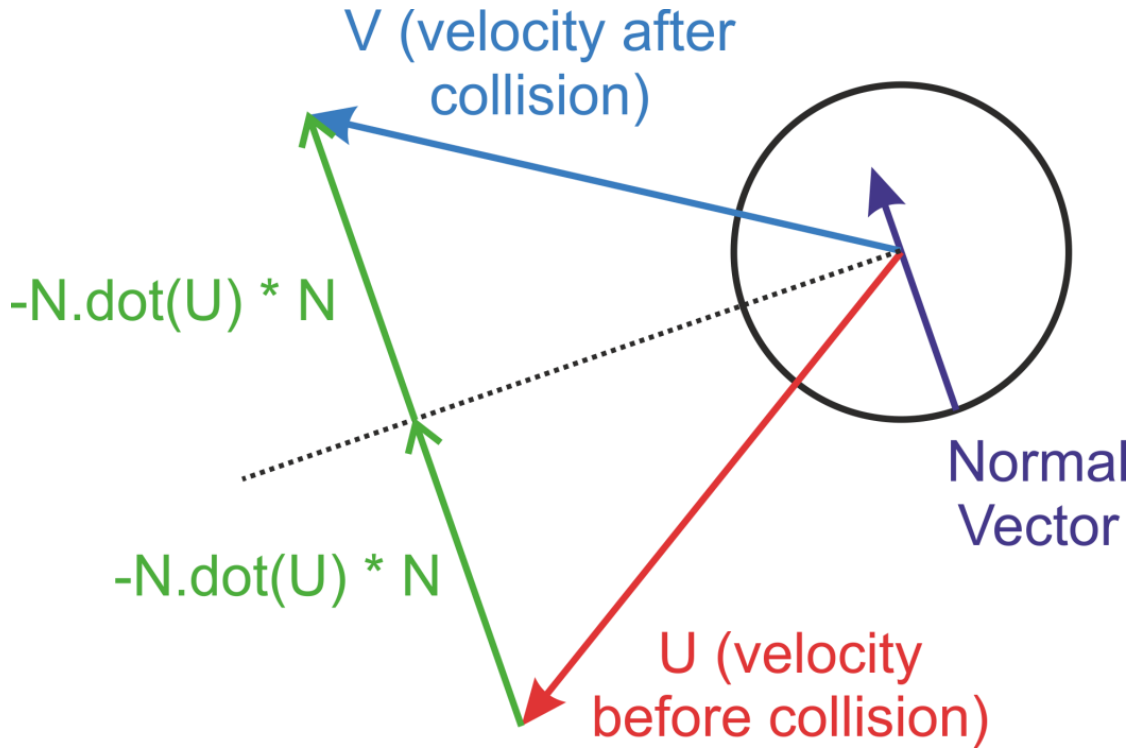
Calculating the reflected velocity

- $\boldsymbol{a}.\boldsymbol{b} = |\boldsymbol{a}||\boldsymbol{b}| \cos\theta$
- $Hyp \times \cos\theta = Adj$
- Angle > 90 so result will be negative
- Could invert the normal instead



Calculating the reflected velocity

- V = U – 2 * N.dot(U) * N



Impulse

- Can either set the velocity to the new velocity
  - This can become tricky with many different objects applying force on one another
- Alternatively we can calculate the equivalent force given the fixed timestep (dt)
  - An ***Impulse*** is the effect of force acting over time to change the momentum of an object
  - $J = F * dt$
  - $F = m * a = m * dv / dt$
  - $J = dt * m * dv / dt = m * dv$

Time to Code

Workshop 3.2 Continue with the sandbox